

Atelier 1 : diagnostic initial

ShopFlow · PostgreSQL 18.6 · résultats et interprétation

Contexte. PostgreSQL 18.6 (Docker postgres:18, WSL2), base et schéma shopflow, chargés par 01_schema.sql et 02_donnees.sql : 1 000 clients, 200 produits, 100 000 commandes, 300 000 lignes. work_mem = 4 MB, shared_buffers = 128 MB. Aucun index ajouté : seuls ceux des contraintes (clés primaires, UNIQUE). Mesures à chaud, le 05/10/2026.

1. Les deux requêtes

Q1 : historique du client 42 (page d'API, très sélective)

```
SELECT id, created_at, total
FROM shopflow.commandes
WHERE client_id = 42
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Q2 : indicateurs mensuels (lit toute la table)

```
SELECT date_trunc('month', created_at) AS mois,
       statut,
       count(*) AS nb_commandes,
       sum(total) AS chiffre_affaires
FROM shopflow.commandes
GROUP BY mois, statut
ORDER BY mois, statut;
```

2. Résultats fonctionnels vérifiés

On vérifie l'exactitude avant de mesurer : ces chiffres sont la référence qui doit rester identique, au centime, après toute optimisation (slides 6 et 24).

Req.	Lignes	Total	Contrôle de référence	OK
Q1	20	14 400,00 €	Le client 42 a 100 commandes (47 760,00 €), la page en garde 20	✓
Q2	30 groupes	29 775 001,25 €	sum(total) sur toute la table : 100 000 commandes, 29 775 001,25 €. Écart 0	✓

3. Mesures : 5 répétitions après échauffement

3 exécutions d'échauffement, puis 5 avec EXPLAIN (ANALYZE, BUFFERS). Execution Time en ms. Les 5 plans de chaque requête sont identiques.

Req.	#1	#2	#3	#4	#5	Médiane	Buffers
Q1	0,207	0,267	0,234	0,208	0,229	0,229	hit=99
Q2	84,3	84,1	80,5	86,2	78,7	84,1	hit=1 680, temp 380/381

Froid ou chaud

Première exécution de Q2, à froid : 1 096,8 ms (shared read=1330). À chaud : ≈ 84 ms (shared read=0). Même plan, 13 fois plus rapide : l'état du cache doit toujours être précisé (slide 11).

4. Nœud coûteux et hypothèse

Q2 : agrégation mensuelle

Nœud (lecture de bas en haut)	Temps propre	Part
Seq Scan on commandes (100 000 lignes)	≈ 25 ms	33 %
Sort : external merge, Disk 3 040 kB	≈ 37 ms	48 %
GroupAggregate (30 groupes)	≈ 15 ms	20 %

Nœud coûteux : le Sort. Il trie 100 000 lignes pour produire 30 groupes et déborde sur disque (381 pages temporaires). Le Seq Scan est normal, puisque toute la table est nécessaire.

Hypothèse : work_mem (4 MB) est trop petit pour ce tri, **et** le planificateur estime 80 048 groupes au lieu de 30, faute de statistique sur date_trunc('month', created_at). Il choisit donc de trier plutôt que de faire un HashAggregate en mémoire. **Testé ensuite sur une copie :** SET work_mem = '32MB' et une statistique sur l'expression, voir « Optimisations testées » (hypothèse confirmée, gain ≈ ×1,7 à ×1,9).

Q1 : historique du client 42

Déjà rapide (≈ 0,23 ms) et l'estimation est exacte (100 lignes prévues, 100 lues). L'accès passe par l'index unique (client_id, cle_idempotence) issu d'une contrainte. **Nœud le plus coûteux : le Bitmap Heap Scan**, qui lit 88 pages pour 100 commandes dispersées, puis un tri n'en garde que 20. **Hypothèse :** le coût croît avec le nombre de commandes du client, parce que l'index ne contient pas created_at. Testé sur une copie : un index sur (client_id, created_at DESC, id DESC) (gain ≈ ×2,9, coût en écriture +44 %, voir « Optimisations testées »).

Ce que j'ai compris de l'atelier 1

- 1. Je vérifie avant de mesurer.** Je compte les lignes et je calcule le total, puis je les compare à une référence calculée autrement. Sans ça, je pourrais accélérer une requête qui renvoie un mauvais résultat sans m'en rendre compte.
- 2. Une mesure ne suffit pas.** Je chauffe la requête puis je la lance 5 fois et je regarde la médiane. La même requête a pris 1 096 ms à froid et 84 ms à chaud : sans échauffement, je comparerais deux situations différentes.
- 3. Je lis le plan de bas en haut.** Chaque ligne est une étape (un nœud). Le temps d'un parent contient déjà celui de ses enfants : pour le coût propre du Sort, je fais $62,6 - 25,4 \approx 37$ ms, et je n'ajoute pas.
- 4. Le coût n'est pas toujours là où on l'attend.** Dans l'agrégation, la lecture de la table est normale (il faut tout lire). Le vrai coût est le tri, qui dépasse les 4 Mo de work_mem et écrit sur le disque.
- 5. Je compare estimé et réel.** PostgreSQL prévoit 80 048 groupes alors qu'il y en a 30 : il se trompe parce qu'il n'a pas de statistique sur date_trunc, et il choisit de trier plutôt que de compter en mémoire.
- 6. Un diagnostic complet** dit quelle étape coûte, pourquoi, et comment le vérifier. Je n'ai rien modifié ici : l'atelier interdit les index, les pistes sont à tester ensuite.

Atelier 2 : une réécriture équivalente

Mêmes données, mêmes règles : on prouve l'équivalence du résultat avant de comparer les durées

Principe (slide 31). Réécrire une requête n'est valable que si le résultat pertinent est identique. Une version plus rapide qui perd ou duplique des lignes est **rejetée**. Cas de test choisis : le **client 42** (100 commandes) et la **commande 42** (3 lignes).

A. Jointure qui duplique les clients, contre EXISTS

Question métier : quels clients ont au moins une commande payée ? Une ligne du résultat doit identifier **un client** (slides 24 et 27).

Avant : jointure (A1)

```
SELECT cl.id, cl.nom
FROM shopflow.clients cl
JOIN shopflow.commandes c ON c.client_id = cl.id
WHERE c.statut = 'payee';
```

Après : EXISTS (A3)

```
SELECT cl.id, cl.nom
FROM shopflow.clients cl
WHERE EXISTS (SELECT 1 FROM shopflow.commandes c
              WHERE c.client_id = cl.id AND c.statut = 'payee');
```

Variante	Lignes	Clients distincts	Verdict
A1 jointure	80 000	800	Rejetée : le client 42 apparaît 100 fois
A2 jointure + DISTINCT	800	800	Correcte, mais construit 80 000 lignes puis déduplique
A3 EXISTS	800	800	Correcte, une ligne par client

Contrôle du contenu : la différence symétrique entre A2 et A3 (EXCEPT dans les deux sens) renvoie **0 ligne**. Les deux listes de clients sont donc identiques.

Variante	#1	#2	#3	#4	#5	Médiane
A1 jointure	27,8	27,4	27,6	28,4	26,4	27,6
A2 + DISTINCT	49,2	40,2	35,3	37,7	42,7	40,2
A3 EXISTS	11,3	13,3	11,7	11,3	10,8	11,3

Durées en ms, 5 répétitions après 3 exécutions d'échauffement.

Interprétation

A3 est la plus rapide (≈ 11 ms contre ≈ 27 ms) et la seule à répondre directement à la question.

PostgreSQL la transforme en Nested Loop Semi Join : pour chaque client, il s'arrête dès qu'il trouve une commande payée (0,8 ligne lue en moyenne par client). La jointure construit 80 000 lignes (Hash Join après un Seq Scan de toute la table). Avec DISTINCT, il faut en plus un HashAggregate : c'est la plus lente. Une sous-requête corrélée n'implique donc pas une exécution complète par ligne (slide 34, question 6).

B. Total par commande sans multiplier commandes.total

Piège (slide 24) : joindre commandes à lignes donne une ligne par **ligne de commande**, pas par commande. Chaque commande a 3 lignes, donc `sum(c.total)` compte chaque total 3 fois.

Avant : faux (B1)

```
SELECT c.id, sum(c.total) AS total          -- FAUX
FROM shopflow.commandes c
JOIN shopflow.lignes l ON l.commande_id = c.id
WHERE c.client_id = 42
GROUP BY c.id;
```

Après : somme des lignes (B2)

```
SELECT c.id, sum(l.qte * l.prix_unitaire) AS total  -- CORRECT
FROM shopflow.commandes c
JOIN shopflow.lignes l ON l.commande_id = c.id
WHERE c.client_id = 42
GROUP BY c.id;
```

Contrôle (client 42)	Nb cmd	Total	Verdict
Référence : commandes.total	100	47 760,00 €	—
B1 sum(c.total)	100	143 280,00 €	Rejetée : total × 3
B2 sum(qte × prix_unitaire)	100	47 760,00 €	Correcte : égale à la référence

Exemple sur la commande 42 : B1 donne **720,00 €** (3 × 240), B2 donne **240,00 €** (58,75 + 80,00 + 101,25), et commandes.total vaut 240,00 €. Comparaison commande par commande de B2 avec commandes.total : **0 écart** sur les 100 commandes.

Variante	#1	#2	#3	#4	#5	Médiane
B1 (faux)	1,15	1,10	1,12	1,10	1,01	1,10
B2 (correct)	2,21	2,01	2,02	2,17	2,05	2,05

Durées en ms, 5 répétitions après échauffement.

Interprétation

B1 est plus rapide (≈ 1,1 ms contre ≈ 2,1 ms), mais son résultat est faux : on la rejette. B1 n'a besoin que de la clé de la ligne : PostgreSQL utilise un Index Only Scan sur lignes, sans lire la table. B2 doit lire qte et prix_unitaire, donc un Index Scan avec accès à la table, puis un Sort avant le regroupement. Le surcoût de B2 (≈ 1 ms) est le prix de l'exactitude : on ne choisit pas une requête uniquement sur sa durée (slide 6).

Conclusion de l'atelier

1. **EXISTS** remplace avantageusement la jointure quand on veut un résultat par client : correct et 2,4 fois plus rapide ici.
2. Pour un total par commande, on **agrége à la bonne granularité** (les lignes), jamais commandes.total après une jointure un-à-plusieurs.
3. Dans les deux cas, l'équivalence a été prouvée par un contrôle de contenu **avant** de comparer les temps. Les plans des variantes sont en annexe.

Ce que j'ai compris de l'atelier 2

1. Réécrire, c'est changer la forme, pas la réponse. Je dois le prouver (mêmes clients, même total) avant de regarder les durées. Pour A, la différence symétrique entre les deux versions vaut 0 ligne.

2. Une jointure avec une table « plusieurs » multiplie les lignes. Un client a 100 commandes : il apparaît 100 fois (80 000 lignes pour 800 clients). Une commande a 3 lignes : son total est compté 3 fois (720 au lieu de 240). Je me demande toujours : « qu'est-ce qu'une ligne de mon résultat ? ».

3. Pour savoir si quelque chose existe, j'utilise EXISTS. Pas de doublons, et PostgreSQL s'arrête dès qu'il trouve : 11 ms contre 27 ms pour la jointure. Condition : je veux un résultat par client, pas une ligne par commande.

4. DISTINCT ne corrige pas le problème, il le masque. Il construit quand même 80 000 lignes puis les supprime : 40 ms, la version la plus lente.

5. Plus rapide ne veut pas dire meilleur. B1 est presque deux fois plus rapide (1,10 ms contre 2,05 ms) mais son total est triplé : je la rejette. B2 coûte 1 ms de plus parce qu'elle lit la table, et c'est le prix de l'exactitude.

6. Je somme à la bonne granularité : les lignes de commande, jamais commandes.total après une jointure un-à-plusieurs.

Optimisations testées : le bilan

Chaque optimisation est mesurée avec le même protocole, avec un résultat identique à l'original

Cadre. L'atelier 1 interdit d'ajouter un index ou une statistique dans la base du laboratoire. Les pistes ont donc été testées sur une **copie** (shopflow_opt, mêmes 100 000 commandes), **un seul changement à la fois**, avec 3 exécutions d'échauffement puis 5 mesures (médiane). La base shopflow du laboratoire n'a pas été touchée. **Équivalence prouvée** : pour chaque variante, l'empreinte md5 du résultat complet est identique à celle de la requête d'origine.

Q2 : agrégation mensuelle (le problème était le tri qui déborde sur le disque)

Optimisation	Ce qui change dans le plan	Médiane	Gain
Référence (copie)	Sort external merge sur disque ; estimation 80 738 groupes	84,6 ms	—
O1 : SET work_mem = '32MB'	HashAggregate en mémoire ; le tri ne porte plus que sur 30 lignes ; estimation toujours fausse (80 738)	50,2 ms	×1,7
O2 : CREATE STATISTICS sur date_trunc, puis ANALYZE	Estimation corrigée : 30 groupes, HashAggregate de 40 kB	45,5 ms	×1,9
O1 + O2	Même plan que O2	47,9 ms	×1,8 (pas mieux que O2 seule)

Mon hypothèse de l'atelier 1 est confirmée. Les deux causes jouaient chacune : donner plus de mémoire ou corriger l'estimation suffit à remplacer le gros tri par un HashAggregate. **O2 corrige la cause** (PostgreSQL se trompait sur le nombre de groupes), **O1 contourne le symptôme** (le tri n'avait plus à déborder). Les deux gains sont du même ordre ($\approx 1,7$ à $1,9$ fois). **Coût** : O1 augmente la mémoire autorisée par opération de tri ou de hachage et par connexion : à réserver à une session ou une requête précise, pas à tout le serveur. O2 a un coût faible (une statistique à maintenir lors des ANALYZE), mais je ne l'ai pas mesuré. **Recommandation** : O2.

Q1 : historique du client 42 (le problème était la lecture de 100 commandes pour en garder 20)

Optimisation	Ce qui change dans le plan	Médiane	Gain
Référence (copie)	Bitmap Heap Scan (85 pages) puis Sort top-N, 98 buffers	0,442 ms	—

Optimisation	Ce qui change dans le plan	Médiane	Gain
O3 : index (client_id, created_at DESC, id DESC)	Index Scan seul, s'arrête après 20 lignes, plus de Sort, 26 buffers	0,154 ms	×2,9

Pourquoi ça marche : l'index range déjà les commandes du client de la plus récente à la plus ancienne, donc PostgreSQL lit exactement 20 entrées dans l'ordre demandé, sans trier. **Coût mesuré** : l'index pèse 3 984 kB (la table : 6 720 kB, soit +59 %) et il ralentit les écritures : insérer 20 000 commandes prend 236 ms sans lui et 339 ms avec (+44 %). **Quand le garder** : si l'historique d'un client est lu bien plus souvent que les commandes ne sont créées, ce qui est le cas d'une API de consultation.

Gains obtenus par réécriture (atelier 2), sans aucun coût d'index

Réécriture	Avant	Après	Gain
Clients ayant une commande payée : EXISTS au lieu de la jointure	27,6 ms (80 000 lignes, doublons)	11,3 ms (800 lignes)	×2,4
Total par commande : somme des lignes au lieu de commandes.total	total faux (× 3)	total correct	exactitude

Ce que j'ai compris des optimisations

- 1. Optimiser, c'est mesurer un gain ET un coût, avec un résultat inchangé.** Chaque ligne du tableau donne les trois.
- 2. On corrige d'abord la cause.** La statistique règle l'erreur d'estimation à l'origine du mauvais plan ; augmenter work_mem masque le symptôme.
- 3. Un index n'est pas gratuit.** Il accélère la lecture du client 42 (× 2,9) mais ralentit les insertions (+44 %) et occupe 59 % de la taille de la table. Je le choisirais si la lecture domine, ce qui est le cas d'une API de consultation.
- 4. Je ne compare que dans le même environnement.** La copie est physiquement plus compacte que la base du laboratoire (840 pages contre 1 674 pour commandes), donc ses temps de référence diffèrent (Q1 : 0,44 ms contre 0,23 ms). Les gains se lisent au sein de la copie.

Questions de compréhension du cours

Réponses avec un exemple et une condition d'application, comme le demande la slide 33.

1. Pourquoi ne faut-il pas additionner les durées de tous les nœuds ?

Parce que le temps d'un parent **inclut** celui de ses enfants : on compterait le même travail plusieurs fois. Exemple : dans l'agrégation, le Sort affiche 62,6 ms mais le Seq Scan en dessous finit à 25,4 ms ; le coût propre du Sort est ≈ 37 ms. Condition : on soustrait la fin de l'enfant, sauf si le nœud tourne en boucle (loops > 1), où les valeurs sont des moyennes par boucle.

2. Quand un parcours séquentiel peut-il être raisonnable ?

Quand la table est petite, ou quand la requête a besoin d'une grande part des lignes. Exemple : l'agrégation mensuelle lit les 100 000 commandes : le Seq Scan est le bon choix, un index n'apporterait rien. Condition : ça ne tient plus si on cherche peu de lignes dans une grande table (client 42 : 100 commandes sur 100 000).

3. Pourquoi une moyenne faible peut-elle masquer un problème ?

Parce que quelques requêtes très lentes sont noyées dans beaucoup de rapides. Le p95 et le p99 montrent cette queue. Exemple : la slide 8 donne (valeurs fictives du cours) p50 = 80 ms mais p99 = 610 ms. Condition : un p95 n'est fiable qu'avec assez d'observations : avec mes 5 mesures, je ne regarde que la médiane.

4. Qu'est-ce qui rend deux benchmarks comparables ?

Mêmes données, même requête, même résultat attendu, même machine, même charge, et un état de cache documenté. On change un facteur à la fois. Exemple : Q2 à froid (1 096 ms) et à chaud (84 ms) ne sont pas comparables entre elles. Condition : ne comparer que des mesures faites dans le même état.

5. Pourquoi la jointure incidents / incident_events change-t-elle la granularité ?

Une jointure un-à-plusieurs donne une ligne par élément du côté « plusieurs ». Exemple chez moi : clients JOIN commandes donne une ligne par commande (80 000), pas par client (800), et commandes JOIN lignes fait compter chaque total 3 fois. Condition : pour garder un résultat parent, on utilise EXISTS, on agrège avant de joindre, ou on somme au niveau des lignes.

6. Une sous-requête corrélée produit-elle toujours une exécution par ligne ?

Non : le planificateur peut la transformer en semi-jointure. Exemple : mon EXISTS (A3) a donné un Nested Loop Semi Join qui s'arrête dès qu'une commande payée est trouvée (11 ms, plus rapide que la jointure). Condition : on le vérifie dans EXPLAIN (ANALYZE, BUFFERS), on ne le suppose pas.

Annexe 1 : plans EXPLAIN (ANALYZE, BUFFERS) complets (mesure n°5)

Q1 : historique du client 42

```
Limit (cost=329.84..329.89 rows=20 width=22) (actual time=0.183..0.186 rows=20.00 loops=1)
  Buffers: shared hit=99
  -> Sort (cost=329.84..330.09 rows=100 width=22) (actual time=0.182..0.184 rows=20.00 loops=1)
    Sort Key: created_at DESC, id DESC
    Sort Method: top-N heapsort Memory: 27kB
    Buffers: shared hit=99
    -> Bitmap Heap Scan on commandes (cost=5.07..327.18 rows=100 width=22) (actual
        time=0.039..0.147 rows=100.00 loops=1)
      Recheck Cond: (client_id = 42)
      Heap Blocks: exact=88
      Buffers: shared hit=93
      -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04
          rows=100 width=0) (actual time=0.022..0.023 rows=100.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=5

Planning:
  Buffers: shared hit=125
Planning Time: 0.485 ms
Execution Time: 0.229 ms
```

Q2 : agrégation mensuelle

```
GroupAggregate (cost=13279.82..15730.54 rows=80048 width=54) (actual time=54.107..77.900 rows=30.00
 loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=1680, temp read=380 written=381
  -> Sort (cost=13279.82..13529.82 rows=100000 width=20) (actual time=53.828..62.580 rows=100000.00
 loops=1)
    Sort Key: (date_trunc('month'::text, created_at)), statut
    Sort Method: external merge Disk: 3040kB
    Buffers: shared hit=1680, temp read=380 written=381
    -> Seq Scan on commandes (cost=0.00..2924.00 rows=100000 width=20) (actual time=0.760..25.395
        rows=100000.00 loops=1)
      Buffers: shared hit=1674

Planning:
  Buffers: shared hit=152
Planning Time: 0.490 ms
Execution Time: 78.664 ms
```

Annexe 2 : plans de l'atelier 2 (mesure n°5)

A1 : jointure

```
Hash Join (cost=32.50..3168.22 rows=80317 width=18) (actual time=0.986..23.875 rows=80000.00 loops=1)
  Hash Cond: (c.client_id = cl.id)
  Buffers: shared hit=1684
  -> Seq Scan on commandes c (cost=0.00..2924.00 rows=80317 width=8) (actual time=0.721..11.527
        rows=80000.00 loops=1)
        Filter: (statut = 'payee'::text)
        Rows Removed by Filter: 20000
        Buffers: shared hit=1674
  -> Hash (cost=20.00..20.00 rows=1000 width=18) (actual time=0.249..0.250 rows=1000.00 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 58kB
        Buffers: shared hit=10
        -> Seq Scan on clients cl (cost=0.00..20.00 rows=1000 width=18) (actual time=0.009..0.095
              rows=1000.00 loops=1)
              Buffers: shared hit=10
Planning:
  Buffers: shared hit=248
Planning Time: 0.655 ms
Execution Time: 26.422 ms
```

A2 : jointure + DISTINCT

```
HashAggregate (cost=3569.81..3579.81 rows=1000 width=18) (actual time=42.510..42.599 rows=800.00
  loops=1)
  Group Key: cl.id, cl.nom
  Batches: 1 Memory Usage: 89kB
  Buffers: shared hit=1684
  -> Hash Join (cost=32.50..3168.22 rows=80317 width=18) (actual time=1.102..27.041 rows=80000.00
        loops=1)
        Hash Cond: (c.client_id = cl.id)
        Buffers: shared hit=1684
        -> Seq Scan on commandes c (cost=0.00..2924.00 rows=80317 width=8) (actual time=0.695..12.879
              rows=80000.00 loops=1)
              Filter: (statut = 'payee'::text)
              Rows Removed by Filter: 20000
              Buffers: shared hit=1674
        -> Hash (cost=20.00..20.00 rows=1000 width=18) (actual time=0.387..0.389 rows=1000.00 loops=1)
              Buckets: 1024 Batches: 1 Memory Usage: 58kB
              Buffers: shared hit=10
              -> Seq Scan on clients cl (cost=0.00..20.00 rows=1000 width=18) (actual
                    time=0.008..0.220 rows=1000.00 loops=1)
                    Buffers: shared hit=10
Planning:
  Buffers: shared hit=173
Planning Time: 0.617 ms
Execution Time: 42.705 ms
```

A3 : EXISTS

```
Nested Loop Semi Join (cost=0.29..568.98 rows=1000 width=18) (actual time=0.363..10.758 rows=800.00
  loops=1)
  Buffers: shared hit=21707
  -> Seq Scan on clients cl (cost=0.00..20.00 rows=1000 width=18) (actual time=0.008..0.081
        rows=1000.00 loops=1)
        Buffers: shared hit=10
  -> Index Scan using commandes_client_id_cle_idempotence_key on commandes c (cost=0.29..9.92 rows=80
        width=8) (actual time=0.010..0.010 rows=0.80 loops=1000)
        Index Cond: (client_id = cl.id)
        Filter: (statut = 'payee'::text)
        Rows Removed by Filter: 20
        Index Searches: 1000
        Buffers: shared hit=21697
Planning:
  Buffers: shared hit=248
Planning Time: 0.662 ms
Execution Time: 10.832 ms
```

B1 : sum(c.total), faux

```
HashAggregate (cost=767.18..768.43 rows=100 width=40) (actual time=0.911..0.930 rows=100.00 loops=1)
  Group Key: c.id
  Batches: 1 Memory Usage: 56kB
  Buffers: shared hit=394
  -> Nested Loop (cost=5.49..765.68 rows=300 width=14) (actual time=0.055..0.803 rows=300.00 loops=1)
    Buffers: shared hit=394
    -> Bitmap Heap Scan on commandes c (cost=5.07..327.18 rows=100 width=14) (actual
        time=0.040..0.156 rows=100.00 loops=1)
      Recheck Cond: (client_id = 42)
      Heap Blocks: exact=88
      Buffers: shared hit=93
      -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04
          rows=100 width=0) (actual time=0.027..0.027 rows=100.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=5
    -> Index Only Scan using lignes_pkey on lignes l (cost=0.42..4.35 rows=3 width=8) (actual
        time=0.006..0.006 rows=3.00 loops=100)
      Index Cond: (commande_id = c.id)
      Heap Fetches: 0
      Index Searches: 100
      Buffers: shared hit=301

Planning:
  Buffers: shared hit=179
Planning Time: 0.593 ms
Execution Time: 1.007 ms
```

B2 : sum(lignes), correct

```
GroupAggregate (cost=1854.26..1859.26 rows=100 width=40) (actual time=1.877..1.995 rows=100.00 loops=1)
  Group Key: c.id
  Buffers: shared hit=696
  -> Sort (cost=1854.26..1855.01 rows=300 width=18) (actual time=1.866..1.880 rows=300.00 loops=1)
    Sort Key: c.id
    Sort Method: quicksort Memory: 36kB
    Buffers: shared hit=696
    -> Nested Loop (cost=5.49..1841.92 rows=300 width=18) (actual time=0.060..1.757 rows=300.00
        loops=1)
      Buffers: shared hit=693
      -> Bitmap Heap Scan on commandes c (cost=5.07..327.18 rows=100 width=8) (actual
          time=0.037..0.167 rows=100.00 loops=1)
        Recheck Cond: (client_id = 42)
        Heap Blocks: exact=88
        Buffers: shared hit=93
        -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04
            rows=100 width=0) (actual time=0.023..0.024 rows=100.00 loops=1)
          Index Cond: (client_id = 42)
          Index Searches: 1
          Buffers: shared hit=5
      -> Index Scan using lignes_pkey on lignes l (cost=0.42..15.12 rows=3 width=18) (actual
          time=0.010..0.015 rows=3.00 loops=100)
        Index Cond: (commande_id = c.id)
        Index Searches: 100
        Buffers: shared hit=600

Planning:
  Buffers: shared hit=182
Planning Time: 0.558 ms
Execution Time: 2.051 ms
```

Annexe 3 : plans des optimisations (copie, mesure n°5)

Q2 référence (copie)

```
GroupAggregate (cost=12445.82..14906.89 rows=80738 width=54) (actual time=54.555..78.490 rows=30.00 loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=846, temp read=380 written=381
  -> Sort (cost=12445.82..12695.82 rows=100000 width=20) (actual time=54.310..63.319 rows=100000.00 loops=1)
    Sort Key: (date_trunc('month'::text, created_at)), statut
    Sort Method: external merge Disk: 3040kB
    Buffers: shared hit=846, temp read=380 written=381
    -> Seq Scan on commandes (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.014..24.472 rows=100000.00 loops=1)
      Buffers: shared hit=840
Planning:
  Buffers: shared hit=136
Planning Time: 0.350 ms
Execution Time: 79.261 ms
```

Q2 avec O1 : work_mem = 32MB

```
Sort (cost=10881.60..11083.45 rows=80738 width=54) (actual time=50.533..50.535 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=846
  -> HashAggregate (cost=3090.00..4301.07 rows=80738 width=54) (actual time=50.148..50.475 rows=30.00 loops=1)
    Group Key: date_trunc('month'::text, created_at), statut
    Batches: 1 Memory Usage: 2089kB
    Buffers: shared hit=840
    -> Seq Scan on commandes (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.019..26.247 rows=100000.00 loops=1)
      Buffers: shared hit=840
Planning:
  Buffers: shared hit=136
Planning Time: 0.329 ms
Execution Time: 51.517 ms
```

Q2 avec O2 : statistique sur date_trunc

```
Sort (cost=3091.19..3091.26 rows=30 width=54) (actual time=45.145..45.147 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=846
  -> HashAggregate (cost=3090.00..3090.45 rows=30 width=54) (actual time=45.101..45.108 rows=30.00 loops=1)
    Group Key: date_trunc('month'::text, created_at), statut
    Batches: 1 Memory Usage: 40kB
    Buffers: shared hit=840
    -> Seq Scan on commandes (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.015..23.250 rows=100000.00 loops=1)
      Buffers: shared hit=840
Planning:
  Buffers: shared hit=147
Planning Time: 0.407 ms
Execution Time: 45.225 ms
```

Q1 référence (copie)

```
Limit (cost=293.26..293.31 rows=20 width=22) (actual time=0.375..0.378 rows=20.00 loops=1)
  Buffers: shared hit=98
  -> Sort (cost=293.26..293.51 rows=100 width=22) (actual time=0.374..0.375 rows=20.00 loops=1)
    Sort Key: created_at DESC, id DESC
    Sort Method: top-N heapsort Memory: 27kB
    Buffers: shared hit=98
    -> Bitmap Heap Scan on commandes (cost=5.19..290.60 rows=100 width=22) (actual time=0.071..0.307 rows=100.00 loops=1)
      Recheck Cond: (client_id = 42)
      Heap Blocks: exact=85
      Buffers: shared hit=92
      -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.17 rows=100 width=0) (actual time=0.051..0.051 rows=100.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=125
Planning Time: 0.325 ms
Execution Time: 0.442 ms
```

Q1 avec O3 : index composé

```
Limit (cost=0.42..77.57 rows=20 width=22) (actual time=0.042..0.120 rows=20.00 loops=1)
  Buffers: shared hit=26
  -> Index Scan using idx_cmd_client_date on commandes (cost=0.42..386.17 rows=100 width=22) (actual
        time=0.041..0.117 rows=20.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=26
Planning:
  Buffers: shared hit=148
Planning Time: 0.452 ms
Execution Time: 0.157 ms
```